uridah and soumith Add nn.Bilinear and tests 75f1989 · 8 years ago

84 lines (62 loc) · 2.97 KB .

- Code
- Blame

Raw

```
3
4
5   class Linear(Function):
6
7       def forward(self, input, weight, bias=None):
8           self.save_for_backward(input, weight, bias)
9           output = input.new(input.size(0), weight.size(0))
10          output.addmm_(0, 1, input, weight.t())
11          if bias is not None:
12              # cuBLAS doesn't support 0 strides in sger, so we can't use expand
13              self.add_buffer = input.new(input.size(0)).fill_(1)
14              output.addr_(self.add_buffer, bias)
15          return output
16
17      def backward(self, grad_output):
18          input, weight, bias = self.saved_tensors
19
20          grad_input = grad_weight = grad_bias = None
21          if self.needs_input_grad[0]:
22              grad_input = torch.mm(grad_output, weight)
23          if self.needs_input_grad[1]:
24              grad_weight = torch.mm(grad_output.t(), input)
25          if bias is not None and self.needs_input_grad[2]:
26              grad_bias = torch.mv(grad_output.t(), self.add_buffer)
27
28          if bias is not None:
```

```
            return grad_input, grad_weight, grad_bias
        else:
            return grad_input, grad_weight


class Bilinear(Function):

    def forward(self, input1, input2, weight, bias=None):
        self.save_for_backward(input1, input2, weight, bias)

        output = input1.new(input1.size(0), weight.size(0))

        buff = input1.new()

        # compute output scores:
        for k, w in enumerate(weight):
            torch.mm(input1, w, out=buff)
            buff.mul_(input2)
            torch.sum(buff, 1, out=output.narrow(1, k, 1))

        if bias is not None:
            output.add_(bias.expand_as(output))

        return output

    def backward(self, grad_output):
        input1, input2, weight, bias = self.saved_tensors
        grad_input1 = grad_input2 = grad_weight = grad_bias = None

        buff = input1.new()

        if self.needs_input_grad[0] or self.needs_input_grad[1]:
            grad_input1 = torch.mm(input2, weight[0].t())
            grad_input1.mul_(grad_output.narrow(1, 0, 1).expand(grad_input1.size()))
            grad_input2 = torch.mm(input1, weight[0])
            grad_input2.mul_(grad_output.narrow(1, 0, 1).expand(grad_input2.size()))

            for k in range(1, weight.size(0)):
                torch.mm(input2, weight[k].t(), out=buff)
                buff.mul_(grad_output.narrow(1, k, 1).expand(grad_input1.size()))
                grad_input1.add_(buff)

                torch.mm(input1, weight[k], out=buff)
                buff.mul_(grad_output.narrow(1, k, 1).expand(grad_input2.size()))
                grad_input2.add_(buff)

        if self.needs_input_grad[2]:
            # accumulate parameter gradients:
            for k in range(weight.size(0)):
                torch.mul(input1, grad_output.narrow(1, k, 1).expand_as(input1), out=buff)
            grad_weight = torch.mm(buff.t(), input2)

        if bias is not None and self.needs_input_grad[3]:
            grad_bias = grad_output.sum(0)
```

```
83
84            return grad_input1, grad_input2, grad_weight, grad_bias
```